Manual de usuario — Cantera

Cantera administra los equipos, jugadores y resultados de un torneo infantil. El sistema permite consultar las fichas de los jugadores y generar reportes para el torneo.

Antes de comenzar

Necesitas la dirección web que te proporcione quien administra el sistema. Si lo ejecutas en tu computadora, sigue primero la instalación del README y abre `http://127.0.0.1:8000`.

> La aplicación todavía no tiene inicio de sesión. Si se publica en Internet, usa únicamente datos ficticios hasta que se proteja el acceso. Los nombres, fechas de nacimiento y fotografías de menores son datos personales.

Recorrido recomendado

1. Registrar equipos

1. Abre **Equipos** desde el menú.
2. Escribe el nombre del equipo y selecciona **Agregar equipo**.
3. Los equipos se pueden borrar solo cuando todavía no tienen jugadores asociados.

2. Crear las jornadas

1. Abre **Jornadas**.
2. Indica el número de jornada, la fecha del partido y, si se necesita, una descripción.
3. Selecciona **Crear jornada**.
4. No se puede borrar una jornada que ya tenga goles registrados.

Conviene registrar las jornadas antes de capturar los goles o generar la lista arbitral.

3. Registrar y consultar jugadores

1. En **Jugadores**, selecciona **Registrar jugador**.
2. Completa nombres, apellidos, equipo y fecha de nacimiento.
3. Si tienes autorización para usarla, puedes cargar una fotografía JPG, JPEG, PNG o WEBP de hasta 5 MB. La fotografía es opcional en el sistema.

4. Selecciona **Guardar jugador**. La ficha muestra el equipo, la fecha de nacimiento, los goles y las incidencias.
5. Usa **Editar jugador** en la ficha para modificar la información o reemplazar la fotografía.

En la lista de jugadores puedes filtrar por equipo y buscar por nombre o apellido. Desde cada fila puedes abrir la ficha.

4. Registrar goles

1. Abre **Goles** y selecciona **Registrar goles**.
2. Elige el jugador y la jornada; indica la cantidad anotada (de 0 a 99).
3. Guarda el registro.

Solo se permite un registro de goles por jugador y jornada. Si necesitas corregirlo, elimina el registro existente y vuelve a capturarlo. La eliminación no se puede deshacer.

5. Registrar incidencias

1. Abre **Incidencias** y selecciona **Registrar incidencia**.
2. Elige el jugador, describe lo ocurrido, selecciona tarjeta amarilla o roja e indica la fecha.
3. Para una tarjeta roja, también debes indicar hasta qué fecha dura la suspensión. La fecha final debe ser igual o posterior a la incidencia.
4. Guarda el registro.

La lista arbitral considera al jugador suspendido desde la fecha de la incidencia hasta la fecha final, ambas inclusive.

Consultar, imprimir y descargar reportes

En el menú **Reportes** están disponibles:

- **Lista arbitral:** elige una jornada para calcular quién está habilitado o suspendido. Puedes limitarla a un equipo. Incluye jugador, fecha de nacimiento, equipo y fotografía si se registró.
- **Reporte de incidencias:** filtra por equipo, jugador o tipo de tarjeta; selecciona **Filtrar** para actualizar los resultados.
- **Tabla de goleadores:** totaliza los goles y ordena a los jugadores de mayor a menor.

Una vez que veas el resultado correcto:

- Selecciona **Descargar PDF** para bajar un archivo PDF con los resultados filtrados.

- Selecciona **Imprimir** para abrir el diálogo de impresión del navegador. Desde ese diálogo también puedes elegir **Guardar como PDF**.

Datos de demostración

Al instalar el proyecto, se puede cargar un torneo ficticio con cuatro equipos, jugadores, jornadas, goles e incidencias:

```
php artisan db:seed --class=DemoTournamentSeeder
```

Los registros de ejemplo son inventados y no incluyen fotografías. Puedes cargar fotografías autorizadas desde la ficha de cada jugador antes de mostrar los reportes. El seeder evita duplicar sus propios equipos, jornadas, jugadores, goles e incidencias si lo ejecutas de nuevo.

Recomendaciones

- Revisa dos veces las fechas y los filtros antes de imprimir o descargar un reporte.
- Usa únicamente fotografías para las que tengas autorización.
- Antes de eliminar jugadores, goles o incidencias, verifica que realmente deban quitarse: esas eliminaciones no se pueden deshacer desde la aplicación.
- Pide al administrador una copia de respaldo si necesitas conservar la información del torneo.
- No compartas públicamente datos reales de menores. El sistema no tiene cuentas de usuario ni permisos de acceso.

Manual técnico y guía de publicación — Cantera

1. Qué es y qué hace el sistema

Cantera es una aplicación web para administrar un torneo infantil: equipos, jornadas, jugadores, goles, incidencias y tres reportes. Está construida con Laravel 12, PHP 8.2 o superior, MySQL/MariaDB y plantillas Blade. DomPDF genera los archivos PDF descargables.

Este manual explica el proyecto en lenguaje sencillo para quien todavía no conoce Laravel, cómo preparar una instalación y qué se debe revisar antes de publicar una URL para evaluación.

2. Laravel explicado sin experiencia previa

Laravel es el framework PHP que recibe cada solicitud del navegador y coordina lo que ocurre después. En este proyecto el recorrido suele ser:

1. **Ruta (`routes/web.php`):** relaciona una dirección como `/jugadores` o `/reportes/goleadores/pdf` con una acción.
2. **Controlador (`app/Http/Controllers/TournamentController.php`):** valida los formularios, consulta los datos necesarios y decide qué respuesta devolver.
3. **Modelo (`app/Models`):** representa una tabla y sus relaciones. Por ejemplo, un jugador pertenece a un equipo y puede tener goles e incidencias.
4. **Base de datos y migraciones (`database/migrations`):** las migraciones describen las tablas y sus restricciones para que Laravel pueda crearlas.
5. **Vista Blade (`resources/views`):** combina HTML con los datos que el controlador le entrega para construir la página. Las vistas `resources/views/reports/pdf` preparan la versión que se convierte en PDF.

Por ejemplo, al abrir la lista arbitral, Laravel identifica la ruta `reports.referee`, llama al método del controlador, consulta los jugadores y las suspensiones para la jornada elegida y renderiza `reports/referee.blade.php`. Al descargar el PDF se reutilizan esos mismos datos y se renderiza una plantilla PDF con DomPDF.

Conceptos que aparecen en el código

- **Eloquent** es el acceso a la base de datos de Laravel. `Player::with('team')` busca jugadores y carga sus equipos relacionados.
- **Validación** comprueba los formularios antes de guardar; por ejemplo, que un jugador pertenezca a un equipo existente y que la fecha de nacimiento sea válida.
- **Blade** es el motor de plantillas. Expresiones como `{{ $player->first_name }}` muestran valores escapados de forma segura.
- **Rutas con nombre** como `reports.incidents` permiten generar enlaces usando `route('reports.incidents')` sin escribir la URL a mano.
- **Seeder** inserta datos de demostración. `DemoTournamentSeeder` crea registros ficticios que se pueden volver a insertar sin duplicar las mismas filas.
- **Pruebas** en `tests/Feature` simulan solicitudes web para comprobar los flujos, incluida la generación de PDF.

Archivos importantes

Ubicación	Responsabilidad
`routes/web.php`	Direcciones y nombres de las páginas y acciones.
`app/Http/Controllers/TournamentController.php`	Consultas, validación, reglas de negocio y respuesta de reportes.
`app/Models`	Modelos y relaciones de equipos, jornadas, jugadores, goles e incidencias.
`database/migrations`	Estructura de tablas, llaves foráneas e índices.
`database/seeders/DemoTournamentSeeder.php`	Datos ficticios para demostraciones.
`resources/views`	Páginas de la aplicación y plantillas de reportes y PDF.
`storage/app/public/players`	Fotografías que se suben a los perfiles.
`tests/Feature/TournamentWorkflowTest.php`	Pruebas de los principales flujos.

3. Preparación local en Windows con XAMPP

Requisitos

- Windows 10/11 y XAMPP con PHP 8.2 o superior.

- Extensiones PHP `pdo_mysql`, `fileinfo`, `mbstring`, `dom`, `xml` y `gd` (GD se necesita para incluir fotografías en los PDF).
- Composer 2 y un navegador.
- Para las pruebas, la extensión `pdo_sqlite`.
- MySQL/MariaDB iniciado desde el panel de XAMPP.

Comprueba las extensiones con `php -m`. En este entorno `gd` no está activa; habilítala en `C:\xampp\php\php.ini` (por ejemplo, quitando `;` de `extension=gd`) y reinicia la terminal y Apache antes de generar reportes con fotografías.

Instalación

Desde PowerShell:

```
cd C:\xampp\htdocs\proyectoFinal_devWeb
composer install
Copy-Item .env.example .env
```

Si ya existe `.env`, **no lo sobrescribas**; edita el archivo existente. Crea una base de datos llamada `torneo_infantil` en phpMyAdmin y configura, al menos, estos valores en `.env`:

```
APP_NAME=Cantera
APP_ENV=local
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=torneo_infantil
DB_USERNAME=root
DB_PASSWORD=
```

Después ejecuta:

```
php artisan key:generate
php artisan migrate
php artisan storage:link
php artisan db:seed --class=DemoTournamentSeeder
php artisan serve
```

Abre la URL que muestra Laravel, normalmente `http://127.0.0.1:8000`. Mantén la terminal abierta mientras uses el servidor. Para cargar además el usuario de ejemplo incluido por Laravel, puedes ejecutar `php artisan db:seed`; ese usuario no es una cuenta de acceso porque el proyecto aún no implementa autenticación.

`php artisan migrate` crea las tablas. Evita `php artisan migrate:fresh` en una base que contenga datos que quieras conservar: ese comando borra todas las tablas antes de recrearlas.

Pruebas

```
php artisan test
```

Las pruebas usan una base SQLite temporal y no deben borrar ni alterar la base MySQL del torneo.

4. Publicar la aplicación para evaluación

La publicación requiere una cuenta de hosting; este repositorio no está publicado automáticamente y aquí no se han configurado un dominio, servidor ni credenciales. Para que se pueda revisar desde cualquier lugar necesitas un hosting público y HTTPS. Un servicio compartido con cPanel puede ser suficiente si permite ejecutar Laravel 12 y cambiar el document root del dominio; un VPS es otra opción, pero requiere administrar el servidor.

Requisitos del hosting

- PHP 8.2 o superior, extensiones requeridas por Laravel, `pdo_mysql` y `gd` para procesar las fotos incluidas en los PDF.
- Composer 2, MySQL/MariaDB y acceso SSH o una herramienta equivalente para ejecutar Artisan.
- Un dominio o subdominio público con certificado HTTPS.
- Un servidor web cuyo **document root** pueda apuntar a la carpeta `public` de Laravel.
- Permisos de escritura para el usuario del servidor en `storage` y `bootstrap/cache`.

Pasos de publicación

1. **Crea la base de datos** desde el panel del hosting y crea un usuario dedicado con contraseña segura y privilegios solamente para esa base. Conserva el host, nombre, usuario y contraseña.

2. **Sube el código** con Git o un ZIP y descomprímelo en una carpeta privada de la cuenta, no directamente dentro de una carpeta pública. No subas `.env`, `vendor` ni datos personales reales al repositorio.

3. **Instala dependencias de producción** en la carpeta del proyecto:

```
composer install --no-dev --optimize-autoloader --no-interaction
```

4. **Crea `.env` en el servidor**, fuera del directorio público, con valores de producción:

```
APP_NAME=Cantera
APP_ENV=production
APP_DEBUG=false
APP_URL=https://tu-dominio.example
DB_CONNECTION=mysql
DB_HOST=HOST_DE_BASE_DE_DATOS
DB_PORT=3306
DB_DATABASE=NOMBRE_DE_BASE
DB_USERNAME=USUARIO_DE_BASE
DB_PASSWORD=CONTRASENA_SEGURA
```

   Sustituye todos los valores de ejemplo por los proporcionados por el hosting. No publiques ni compartas el contenido de `.env`.

5. **Genera la llave una sola vez** en esa instalación y no la cambies mientras haya información cifrada:

```
php artisan key:generate
```

6. **Apunta el dominio a `public`:** en el panel del hosting configura como document root la carpeta `public` de este proyecto. La carpeta que contiene `.env`, `app`, `storage` y `vendor` debe quedar fuera del alcance web. Si tu hosting no permite configurar el document root, elige un hosting compatible con Laravel; no resuelvas esto exponiendo toda la raíz del proyecto.

7. **Crea tablas y prepara el almacenamiento:**

```
php artisan migrate --force
php artisan db:seed --class=DemoTournamentSeeder --force
php artisan storage:link
```

El seeder crea datos inventados para la demostración, no fotografías. `storage:link` permite servir las imágenes que se suban. Si el proveedor no permite enlaces simbólicos, consulta su mecanismo recomendado para almacenamiento público.

8. **Ajusta permisos** para que el usuario del servidor web pueda escribir en `storage` y `bootstrap/cache`. Evita asignar permisos abiertos como `777`.

9. **Apunta el DNS** del dominio o subdominio al hosting según los valores que indique el proveedor. Activa HTTPS en su panel, confirma que `APP_URL` use `https://` y abre la URL pública para comprobar las páginas y la descarga de los tres PDF. No uses `php artisan serve` como servidor de producción.

10. **Comparte la URL** con quien calificará. Usa un subdominio de evaluación si no tienes dominio propio; si necesitas restringir quién ve la demostración, configura autenticación en la aplicación o protección por contraseña en el hosting y comparte las credenciales en privado.

Los comandos Artisan normalmente se ejecutan desde la carpeta privada del proyecto usando la versión PHP seleccionada por el hosting. Algunos proveedores muestran esa ruta en su panel o en la documentación SSH.

Protección de datos antes de publicar

El sistema no tiene inicio de sesión ni autorización: actualmente cualquier visitante que llegue a la URL puede ver, registrar, modificar o borrar información. No cargues nombres, fechas de nacimiento ni fotografías reales de menores en una instancia pública. Antes de usar datos reales, implementa autenticación y permisos y verifica que no se pueda acceder a páginas o reportes sin autorización. Para una revisión pública, conserva solo datos ficticios y fotos ilustrativas con autorización.

Además:

- Mantén `APP_DEBUG=false` y la base de datos inaccesible desde el público.
- No publiques `.env`, contraseñas, llaves ni credenciales en Git, manuales o capturas.
- Configura copias de seguridad periódicas de la base de datos **y** de `storage/app/public`; prueba que puedas restaurarlas.

- Limita quién recibe las credenciales del hosting y de la base de datos.

5. Estado frente a los requisitos del proyecto

El PDF de la consigna solicita registrar jugadores, goles e incidencias; generar listas e informes; mostrar una ficha individual; llevar datos precargados; publicar una URL y entregar documentación.

Requisito	Estado en este proyecto
Equipos, jugadores, fechas de nacimiento y carga de foto	Implementado. La fotografía es opcional y solo se muestra si se sube una.
Jornadas, goles por jugador y cantidad	Implementado. Hay como máximo un registro por jugador y jornada.
Incidencias con tarjeta y suspensión por roja	Implementado; el intervalo de suspensión incluye la fecha inicial y final.
Lista arbitral por equipo/jornada y estado de suspensión	Implementado; la foto aparece si existe en el perfil.
Incidencias filtradas por equipo, jugador o tarjeta	Implementado.
Goleadores totalizados y ordenados por goles	Implementado; el PDF también incluye foto cuando está disponible.
Ficha completa del jugador	Implementado con datos personales, equipo, goles e incidencias.
Descargar e imprimir reportes en PDF	Implementado; la descarga conserva los filtros elegidos y la impresión usa el diálogo del navegador.
Datos de demostración	Seeder ficticio disponible. Las fotografías deben cargarse manualmente con imágenes autorizadas.
Publicación con URL accesible desde Internet	Pendiente de ejecutar en un hosting y compartir la URL; este repositorio no contiene una dirección publicada.
Inicio de sesión y control de permisos	Pendiente; es un requisito previo para publicar datos personales reales.

Requisito	Estado en este proyecto
Documento de entrega en PDF y reflexión del grupo	Se genera `DOCUMENTACION_PROYECTO.pdf` desde estos manuales con `php scripts/generate-documentation-pdf.php`. Completa antes la experiencia real del grupo en la sección de reflexión y vuelve a generar el PDF.

6. Recomendaciones para trabajar con Laravel

- Divide el trabajo siguiendo las responsabilidades que ya tiene el proyecto: rutas para direcciones, controladores para el flujo, modelos para datos y Blade para presentación.
- Valida toda entrada y conserva las restricciones de la base de datos; la validación visual por sí sola no protege los datos.
- Escribe pruebas para reglas de negocio y cambios en rutas o reportes antes de publicar.
- Usa migraciones para cambiar el esquema y seeders con datos ficticios para preparar una demostración reproducible.
- En producción no muestres errores internos; usa `APP_DEBUG=false`, HTTPS, secretos fuera del repositorio y respaldos.
- Antes de actualizar Laravel o una dependencia, revisa compatibilidad, ejecuta las pruebas y publica primero en un entorno de prueba.

7. Reflexión del equipo para completar

La consigna también pide comentarios personales sobre la facilidad o dificultad del framework. Completa esta sección con la experiencia real del grupo antes de exportar el documento final:

- Integrantes y tareas realizadas:
- ¿Qué fue lo más sencillo de Laravel en este proyecto?:
- ¿Qué dificultad encontraron y cómo la resolvieron?:
- ¿Qué recomendarían a otro grupo que empieza con Laravel?:

Después de completar la reflexión, regenera el documento listo para imprimir o entregar:

```
php scripts/generate-documentation-pdf.php
```

El comando une este manual y el manual de usuario en `DOCUMENTACION_PROYECTO.pdf`. Conserva ambos Markdown como fuentes editables.